Determination of the Absolute Zero Point

Lukas Mengis, Kiran Suda
06 November 2026

Abstract

abstract

1 Introduction

The absolute zero point is a fundamental fixed point. It has the lowest possible temperature t_0. There is no lower temperature.

The Temperature is also contained in the ideal-gas-law. Here n is the number of particles, k_B is the Boltzmann constant, T is Temperature, p is pressure, and V is the Volume.

$$nk_BT = pV \tag{1.1}$$

2 Experiment

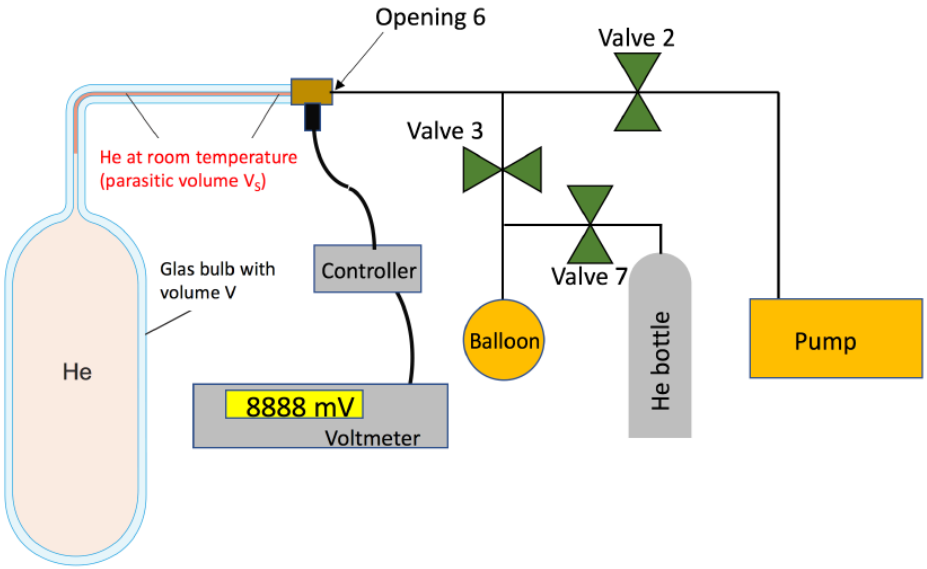


Figure 1: Shows the experimental setup. [1]

3 Results

3.1 Calibration

For the calibration we measured the laboratory temperature t_l. We also measured the reading from the barometer $\tilde{p_l}$. Aswell as the output-signal of the pressure sensor when exposed to laboratory pressure U_l. Then we measured the output-signal at low pressure U_2. This gave:

$$\begin{aligned} t_l &= (22 \pm 1)°C \\ \tilde{p_l} &= (716 \pm 1)\ \text{mmHg} \\ U_l &= (128.0 \pm 0.2)\ \text{mV} \\ U_2 &= (-5.3 \pm 0.2)\ \text{mV} \end{aligned} \tag{3.1}$$

Far all temperatures we estimated $\sigma_t = 1°C$. Since the smallest scale on the Barometer is ± 1 mmHg, we got $\sigma_p = 1$ mmHg. For all output voltages we used $\sigma_U = 0.2$ mV. The σ_U comes from the manual [1] where there is the 0.05% full scale $= 0.075$ mV percicion error and another error due to temperature difference of $0.08\frac{\text{mV}}{°\text{C}}$. Since we estimated $\sigma_t = 1°C$ we got $\sigma_U = 0.075\ \text{mV} + 0.08\ \text{mV} \approx 0.2\ \text{mV}$.

3.2 Absolute Zero

The measurements to determine the absolute zero temperature are the output signals of the pressure sensor. We use U_k for boiling water and U_e for freezing water. The results are:

$$\begin{aligned} U_k &= (127.6 \pm 0.2)\ \text{mV} \\ U_e &= (92.5 \pm 0.2)\ \text{mV} \end{aligned} \tag{3.2}$$

3.3 Liquid Nitrogen

The measurment for liquid Nitrogen (ln2) is U_{ln2}. This is the output signal from the pressure sensor when submerged in boiling liquid Nitrogen.

$$U_{\text{ln2}} = (24.7 \pm 0.2)\ \text{mV} \tag{3.3}$$

4 Data Analysis

4.1 Calibration

From the barometer reading $\tilde{p_l}$ and laboratory temperature t_l we calculate the pressure in the laboratory p_l

$$p_l = (\tilde{p_l} - \text{correction}(t_l)) * \frac{133.322}{100} = ((716 \pm 1) - 3) * \frac{133.322}{100}\ \text{mbar} = (951 \pm 1)\ \text{mbar} \tag{4.1}$$

We used the value of U_k for U_l. So $U_l = (127.6 \pm 0.2)\ \text{mV}$ for all calculations (see Section 5). The value for the low pressure is per assumption [1]:

$$p_2 = (0.1 \pm 0.1)\ \text{mbar} \tag{4.2}$$

From here we fitted a linear function of the form:

$$p(U) = p_0 + CU \tag{4.3}$$

through (p_l, U_l) and (p_2, U_2). We calculated:

$$\begin{aligned} C &= \frac{p_l - p_2}{U_l - U_2} = (7.16 \pm 0.02)\frac{\text{bar}}{V} \\ p_0 &= p_l - \frac{p_l - p_2}{1 - \frac{U_2}{U_l}} = (0.038 \pm 0.001)\ \text{bar} \end{aligned} \tag{4.4}$$

For the error propagations we always used a function of the starting parameters to calculate the values. We used the scipy Jacobian [2] function to calculate all the derivatives. So for C it is:

$$\begin{aligned} z_C &= (p_l, p_2, U_l, U_2) \\ \sigma_{z_C} &= \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}\right) = (0.001\ \text{bar}, 0.0001\ \text{bar}, 0.2\ \text{mV}, 0.2\ \text{mV}) \\ f_C(z_C) &:= \frac{p_l - p_2}{U_l - U_2} \\ \sigma_C &= \sqrt{\left(J_{f_C}(z_C)^2\right)_{1\times 4}\left(\sigma_{z_C}^2\right)_{4\times 1}^T} = 0.02\frac{\text{bar}}{V} \end{aligned} \tag{4.5}$$

Where

$$\begin{aligned} J_{f_C}(z_C)^2 &= \left(\left(\partial_{p_l} f_C(z_C)\right)^2, \left(\partial_{p_2} f_C(z_C)\right)^2, \left(\partial_{U_l} f_C(z_C)\right)^2, \left(\partial_{U_2} f_C(z_C)\right)^2\right) \\ \sigma_{z_C}^2 &= \left(\sigma_{p_l}^2, \sigma_{p_2}^2, \sigma_{U_l}^2, \sigma_{U_2}^2\right) \end{aligned} \tag{4.6}$$

We used the same prcedure for all propagated errors. The exact calculations can be found in the python program at Section A.1. We used AI-tools to help debug the python program. We always made the function depend only on the measured values and the values assumed and not on previously caluclated values. We did this to minimize the effect of correlation. Since previously calculated values can depend on the same starting parameters they correlate. To account for correlation the error propagation gets more complicated and one has to estimate the correlation between the values. This is why we always directly calculated from the original values, so we can assume that they are uncorrelated.

So for p_0 it is the same procedere (shown in Section A.2.ii).

The calibration gives the following calibration-curve:

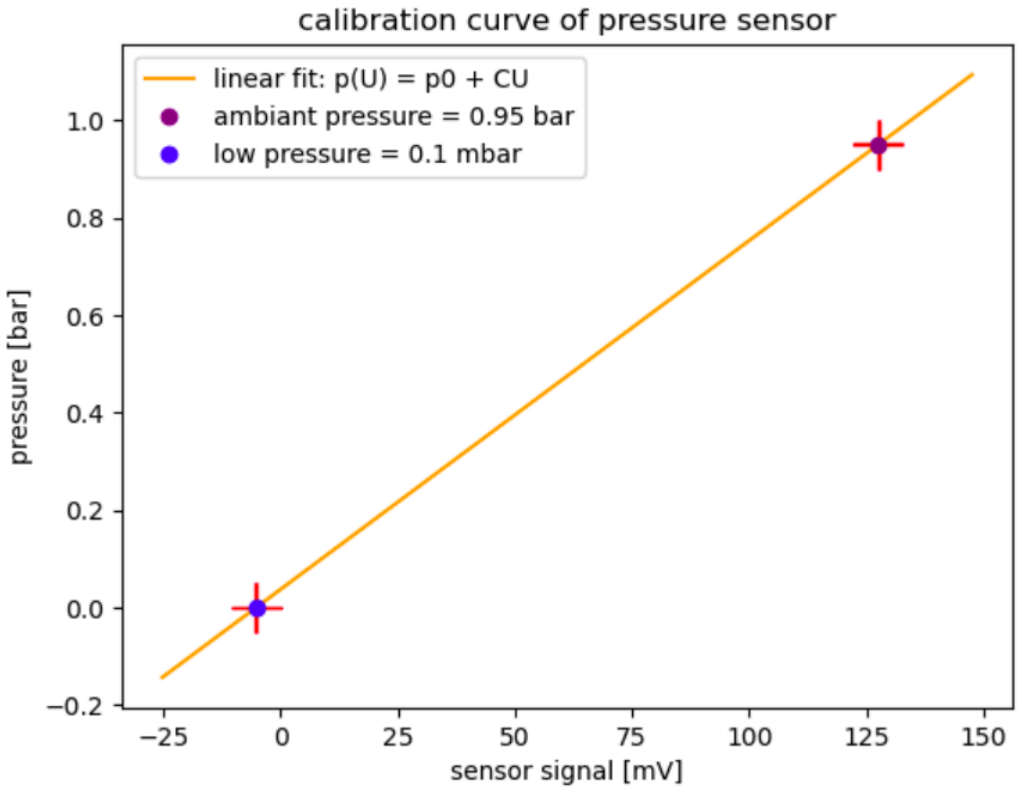


Figure 2: The two calibration points. (p_l, U_l) is purple and (p_2, U_2) is blue. Orange line is $p(U) = p_0 + CU$. Note: The red lines are not the error bars. They are the caps of the error bars. The error bars are smaller than the points and thus barely visible.

4.2 Absolute Zero

From the calibration curve and U_k and U_e we obtain the pressure p_k and p_e at boiling temperature (water) t_k and at freezing temperature t_e.

$$\begin{aligned} p_k &= p(U_k) = (951 \pm 2)\ \text{mbar} \\ p_e &= p(U_e) = (700 \pm 2)\ \text{mbar} \\ t_k &= (98 \pm 1)°C \\ t_e &= (0 \pm 1)°C \end{aligned} \tag{4.7}$$

We obtained t_k through a table in the laboratory where the boiling temperature of water is denoted in dependance of p_l. For the freezing temperature t_e we have $t_e = 0$ per assumption [1]. The error calculation can be found in Section A.2 We can now also plot the pressure/temperature dependance:

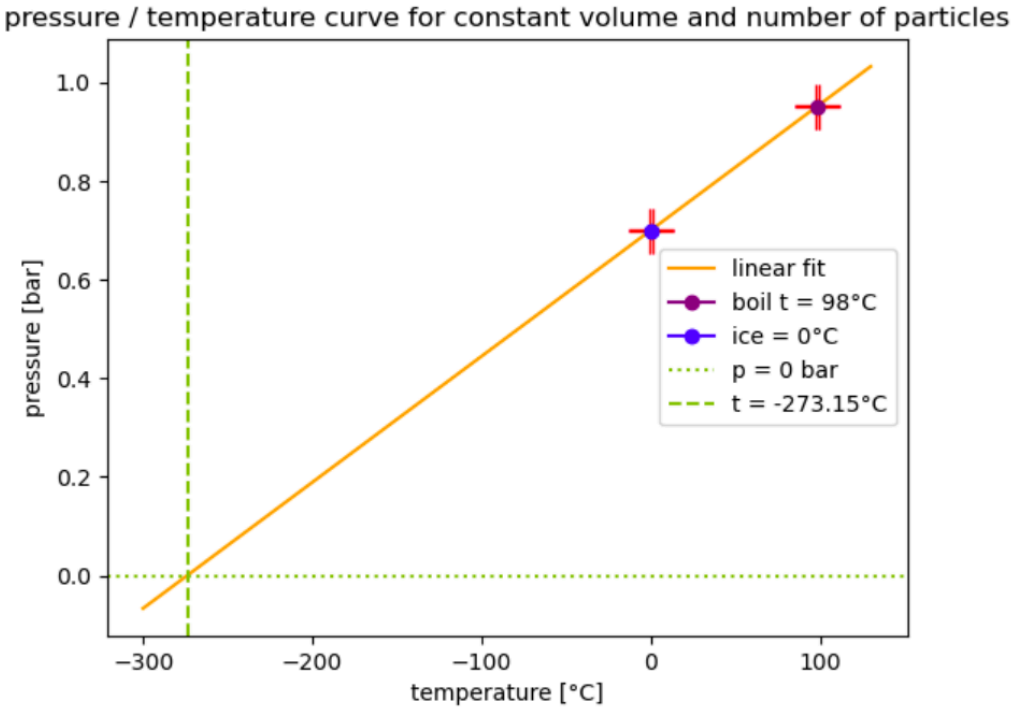


Figure 3: The pruple point is the measured point of (t_k, p_k). The blue point is the measured point of (t_e, p_e). The orange line is a linear fit. From extrapolating the orange curve, one gets t_0'. The green dashed line at $t = -273.15°C$ represents the literature value of absolute zero temperature. It does not represent the experimental value of t_0'. Note: The red lines are not the error bars. They are the caps of the error bars. The error bars are smaller than the points and thus barely visible.

From the obtained values we can caluclated the temperature of absolute zero. t_0' is a simpler approximative calculation. t_0 is more accurate since it takes the parasitic volume V_S into account (see Figure 1).

$$t_0' = -\frac{p_e}{p_k - p_e} t_k = (-274 \pm 3)°C \qquad [1] \tag{4.8}$$

The error calculation can be found in Section A.2. We now define:

$$\begin{gathered} \varepsilon = 10^{-3} \\ \gamma = 10^{-5}\ /°\mathrm{C} \\ a = (1+\varepsilon)p_e - (1+\varepsilon+\gamma t_k)p_k \\ b = \varepsilon(p_k - p_e)t_k + (1+\gamma t_k)p_k t_l - p_e(t_l + t_k) \\ c = p_e t_l t_k \qquad [1] \end{gathered} \tag{4.9}$$

this gives:

$$t_0 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} = (-273 \pm 5)°C \qquad [1] \tag{4.10}$$

The error calulcation can be found in Section A.2.

4.3 Liquid Nitrogen

From the calibration curve and the measurment U_{ln2} we obtain the value for the pressure p_{ln2} at temperature of liquid nitrogen (t_{ln2})

$$p_{\mathrm{ln2}} = (214 \pm 1)\ \mathrm{mBar} \tag{4.11}$$

The error calculation is in Section A.2. Again there is a simpler approximation for the temperature of liquid Nitrogen (t'_{ln2}). Aswell as a more complicated and more accurate value t_{ln2}. We have:

$$t'_{\mathrm{ln2}} = \left(1 - \frac{p_{\mathrm{ln2}}}{p_e}\right) t_0 = (-189 \pm 4)°C \qquad [1] \tag{4.12}$$

The error calculation is in Section A.2. We now define:

$$\begin{aligned} \varepsilon &= 10^{-3} \\ \gamma &= 10^{-5}\ /^{\circ}\mathrm{C} \\ A &= \frac{p_e}{t_e - t_0} + \varepsilon \frac{p_e - p_{\ln 2}}{t_l - t_0} \qquad [1] \end{aligned} \tag{4.13}$$

this gives:

$$t_{\ln 2} = \frac{A t_0 + p_{\ln 2}}{A - \gamma p_{\ln 2}} = (-189 \pm 4)°C \qquad [1] \tag{4.14}$$

The error calculation is in Section A.2.

5 Discussion

Whilst measuring the value for U_l we did not disconnect the pressurized hose from the system. We instead opened the valves to expose the sensor to the remaining pressure in the hose system. This pressure is not the same as the ambiant laboratory pressure. Whilst heating up the helium in the steam bath, the hose was fully disconnected and the valve at position 6 (see in Figure 1) was open. This means that the sensor was exposed to laboratory pressure. We also noticed that the reading of the sensor was stable in this configuration. We therefore concluded that the measurment of U_k better represents the laboratory pressure U_l. We therefore replaced the measured value of $U_l = (128.0 \pm 0.2)\ \mathrm{mV}$ with $U_k = (127.6 \pm 0.2)\ \mathrm{mV}$. So in all calculations: $U_l = U_k = (127.6 \pm 0.2)\ \mathrm{mV}$.

The literature value of the absolute zero temperature ($-273.15°C$) lies within the experimental result range of $t_0 = (-273 \pm 5)°C$.

The literature value of the temperature of liquid Nitrogen is: $-195.8°C$. This value lies outside the experimentally determined range of $t_{\ln 2} = (-189 \pm 4)°C$. So there is likely a systematic error. One possible source of error is leakage of gas. There could be small amounts of gas entering the lower pressure system, which would increase the measured pressure. This would lead to a higher experimentally determined boiling temperature, like in our case. The measurment of the liquid Nitrogen was taken last, so in this case it would not be contradictury to an accurate measurment of absolute zero temperature t_0. Another source could be that the temperature of the helium never fully reached the temperature of liquid Nitrogen. In this case it would be warmer, since we never cooled it more down, than with liquid Nitrogen. This again would lead to a too high experimental value of $t_{\ln 2}$. Also we noticed a drop in output voltage of the pressure sensor, after submerging the glas bulb (Figure 1) in liquid Nitrogen (this is the expected behaviour). After the voltage dropped, it immediatly began to rise again. So the output voltage never stabilized, which made an accurate measurment difficult. We thus think, that an inaccurate measurment is the most likely cause of error.

6 Conclusion

conclusion

A Appendix

A.1 python skript

```
# %%
import numpy as np
import matplotlib.pyplot as plt
from scipy.differentiate import jacobian

# %%
##### calibration
std_fs = 0.15 * 0.05 / 100 # full scale error from manual
std_u_temp = 0.08 / 1000 # another error from manual
std_U = std_fs + std_u_temp # error of U general

# lab pressure calibration point
mmPerHg = 713 #+-1 (mm Hg) with correction for pressure
U_lab = 0.12755 # 0.12795 measured value in lab (but probalby a bit off) # measurement
p_lab = mmPerHg * 133.322 / 100000 # measurement  2.72 mm / Hg  716 reading at 21.5°
std_p_lab = 1 * 133.322 / 100000 # estimation

# low-pressure (# evacuate pump (7-10min))
U_2 = (-1) * 0.00526 # measurement
p_2 = 0.0001 # +- 0.0001 bar (manual)
std_p_2 = 0.0001

print(std_U * 1000)
print('p_lab =', p_lab, '+-', std_p_lab)

# %%
def f_c (data):
    pl = data[0]
    p2 = data[1]
    ul = data[2]
    u2 = data[3]
    return (pl - p2) / (ul - u2)

def f_p0 (data):
    pl = data[0]
    p2 = data[1]
    ul = data[2]
    u2 = data[3]
    return pl - (pl - p2) / (1 - u2 / ul)

def f_pr (data):
    pl = data[0]
    p2 = data[1]
    ul = data[2]
    u2 = data[3]

    u = data[4]

    c = f_c(data[:-1])
    p0 = f_p0(data[:-1])
    return p0 + u * c

# %%
point_c = np.array([p_lab, p_2, U_lab, U_2])
sigma_c = np.array([std_p_lab, std_p_2, std_U, std_U])
point_p0 = point_c
sigma_p0 = sigma_c
C = f_c(point_c)
p_0 = f_p0(point_p0)
```

```
#error prop
jac_c = jacobian(f_c, point_c).df
std_c = np.sqrt(jac_c**2 @ sigma_c.transpose()**2)
jac_p0 = jacobian(f_p0, point_p0).df
std_p0 = np.sqrt(jac_p0**2 @ sigma_p0.transpose()**2)

#print
print('C =', C, '+-', std_c)
print('p_0 =', p_0, '+-', std_p0)

#plot
linspace = np.linspace(U_2 - 0.02, U_lab + 0.02, 20)
points = np.zeros((len(linspace), 5))
points[:,: 4] = point_c
points[:, 4] = linspace
plt.title('calibration curve of pressure sensor')
plt.errorbar(U_lab * 1000, p_lab, std_p_lab, 1000 * std_U, capsize=10, color='r')
plt.errorbar(U_2 * 1000, p_2, std_p_2, 1000 * std_U, capsize=10, color='r')
plt.plot(linspace * 1000, f_pr(points.transpose()), color='orange', label='linear fit: p(U) = p0 +
CU')
plt.plot(U_lab * 1000, p_lab, marker='o', linestyle='', color='purple', label='ambiant pressure =
0.95 bar')
plt.plot(U_2 * 1000, p_2, marker='o', linestyle='', color='b', label='low pressure = 0.1 mbar')
plt.xlabel('sensor signal [mV]')
plt.ylabel('pressure [bar]')
plt.legend()

# %%
##### absolute zero
# evacuate pump (7-10min (maybe already done))
# fill with helium for at least two minutes
# repeat procedure once

# %%
# boiling water (close opening after heating)
U_k = 0.12755 # measurment
t_k = 98.242 # table relation with pressure #+-1
std_t_k = 1 # estimation
point_k = np.array([p_lab, p_2, U_lab, U_2, U_k])
sigma_k = np.array([std_p_lab, std_p_2, std_U, std_U, std_U])

# calculation
p_k = f_pr(point_k)

# error prop
jac_pk = jacobian(f_pr, point_k).df
std_p_k = np.sqrt(jac_pk**2 @ sigma_k.transpose()**2)

print('p_k =', p_k, '+-', std_p_k)

# %%
# ice water
# make cold ice water wait ca 2min
U_e = 0.0925 # measurment
t_e = 0 # temperature ice-water
std_t_e = 1 # estimation
point_e = np.array([p_lab, p_2, U_lab, U_2, U_e])
sigma_e = np.array([std_p_lab, std_p_2, std_U, std_U, std_U])

# calculation
p_e = f_pr(point_e)

# error prop
```

```
jac_pe = jacobian(f_pr, point_e).df
std_p_e = np.sqrt(jac_pe**2 @ sigma_e.transpose()**2)

print('p_e =', p_e, '+-', std_p_e)

temp_samples = np.linspace(-300, 130, 100)
plt.title('pressure / temperature curve for constant volume and number of particles')
plt.errorbar(t_k, p_k, std_p_k, std_t_k, capsize=10, color='r')
plt.errorbar(t_e, p_e, std_p_e, std_t_e, capsize=10, color='r')
plt.plot(temp_samples, temp_samples * (p_k - p_e) / (t_k - t_e) + p_e - (p_k - p_e) / (t_k - t_e) *
t_e, color='orange', label='linear fit')
plt.plot(t_k, p_k, marker='o', color='purple', label='boil t = 98°C')
plt.plot(t_e, p_e, marker='o', color='b', label='ice = 0°C')
plt.axhline(0, linestyle='dotted', color="#95B416", label='p = 0 bar')
plt.axvline(-273.15, linestyle='--', color='#95B416', label='t = -273.15°C')
plt.ylabel('pressure [bar]')
plt.xlabel('temperature [°C]')
plt.legend()

# %%
def f_t0approx (data):
    pl = data[0]
    p2 = data[1]
    ul = data[2]
    u2 = data[3]

    uk = data[4]
    ue = data[5]
    pk = f_pr(data[:5])
    pe = f_pr(data[[0, 1, 2, 3, 5]])

    tk = data[6]

    return (-1) * pe * tk / (pk - pe)

# %%
##### calculations:
point_t0approx = np.array([p_lab, p_2, U_lab, U_2, U_k, U_e, t_k])
sigma_t0approx = np.array([std_p_lab, std_p_2, std_U, std_U, std_U, std_U, std_t_k])

# calculation
t_0_approx = f_t0approx(point_t0approx)

# error prop
jac_t0approx = jacobian(f_t0approx, point_t0approx).df
std_t_0_approx = np.sqrt(jac_t0approx**2 @ sigma_t0approx.transpose()**2)



print('t_0_approx =', t_0_approx, '+-', std_t_0_approx)

# %%
epsilon = 10**(-3)
gamma = 10**(-5)

def f_t0(data):
    pl = data[0]
    p2 = data[1]
    ul = data[2]
    u2 = data[3]

    uk = data[4]
    ue = data[5]
```

```
    pk = f_pr(data[:5])
    pe = f_pr(data[[0, 1, 2, 3, 5]])

    tk = data[6]
    te = data[7]
    tl = data[8]

    aa = (1 + epsilon) * pe - (1 + epsilon + gamma * tk) * pk
    bb = epsilon * (pk - pe) * (te + tk) + (1 + gamma * tk) * pk * (te + tl) - pe * (tl + tk)
    cc = pe * tl * tk + epsilon * (pe - pk) * te * tk - (1 + gamma * tk) * pk * te * tl

    return ((-1) * bb + np.sqrt(bb**2 - 4 * aa * cc)) / (2 * aa)


# %%
t_lab = 21.5 # measurment +-1
std_t_lab = 1 # estimation
point_t0 = np.array([p_lab, p_2, U_lab, U_2, U_k, U_e, t_k, t_e, t_lab])
simga_t0 = np.array([std_p_lab, std_p_2, std_U, std_U, std_U, std_U, std_t_k, std_t_e, std_t_lab])

# calculation
t_0 = f_t0(point_t0)

# error prop
jac_t0 = jacobian(f_t0, point_t0).df
std_t_0 = np.sqrt(jac_t0**2 @ simga_t0.transpose()**2)

print('t_0 =', t_0, '+-', std_t_0)

# %%
##### liquid nitrogen
U_LN2 = 0.02465 # measurement
point_ln2 = np.array([p_lab, p_2, U_lab, U_2, U_LN2])
sigma_ln2 = np.array([std_p_lab, std_p_2, std_U, std_U, std_U])

# calculation
p_LN2 = f_pr(point_ln2)

# error prop
jac_pln2 = jacobian(f_pr, point_ln2).df
std_p_LN2 = np.sqrt(jac_pln2**2 @ sigma_ln2.transpose()**2)

print('p_LN2 =', p_LN2, '+-', std_p_LN2)

# %%
def f_tln2approx (data):
    pl = data[0]
    p2 = data[1]
    ul = data[2]
    u2 = data[3]

    uk = data[4]
    ue = data[5]
    pe = f_pr(data[[0, 1, 2, 3, 5]])

    tk = data[6]
    te = data[7]
    tl = data[8]

    t0 = f_t0(data[0:9])

    uln2 = data[9]
    pln2 = f_pr(data[[0, 1, 2, 3, 9]])
```

```
    return (1 - pln2 / pe) * t0


# %%
point_tln2approx = np.array([p_lab, p_2, U_lab, U_2, U_k, U_e, t_k, t_e, t_lab, U_LN2])
sigma_tln2approx = np.array([std_p_lab, std_p_2, std_U, std_U, std_U, std_U, std_t_k, std_t_e,
std_t_lab, std_U])

# calculation
t_LN2_approx = f_tln2approx(point_tln2approx)

# error prop
jac_tln2approx = jacobian(f_tln2approx, point_tln2approx).df
std_t_LN2_approx = np.sqrt(jac_tln2approx**2 @ sigma_tln2approx.transpose()**2)

print('t_LN2_approx =', t_LN2_approx, '+-', std_t_LN2_approx)

# %%
def f_tln2 (data):
    pl = data[0]
    p2 = data[1]
    ul = data[2]
    u2 = data[3]

    uk = data[4]
    ue = data[5]
    pe = f_pr(data[[0, 1, 2, 3, 5]])

    tk = data[6]
    te = data[7]
    tl = data[8]

    t0 = f_t0(data[0:9])

    uln2 = data[9]
    pln2 = f_pr(data[[0, 1, 2, 3, 9]])

    aaaaa = pe / (te - t0) + epsilon * (pe - pln2) / (tl - t0)

    return (aaaaa * t0 + pln2) / (aaaaa - gamma * pln2)


# %%
point_tln2 = np.array([p_lab, p_2, U_lab, U_2, U_k, U_e, t_k, t_e, t_lab, U_LN2])
sigma_tln2 = np.array([std_p_lab, std_p_2, std_U, std_U, std_U, std_U, std_t_k, std_t_e, std_t_lab,
std_U])

# calculation
t_LN2 = f_tln2(point_tln2)

# error prop
jac_tln2 = jacobian(f_tln2, point_tln2).df
std_t_LN2 = np.sqrt(jac_tln2**2 @ sigma_tln2.transpose()**2)

print('t_LN2 =', t_LN2, '+-', std_t_LN2)
```

A.2 Error Propagation

An overline $\overline{a}$ means in this section: $\overline{a} = f_a(z_a)$. So $\overline{a}$ is the value a but as a function dependant on z_a. The vector z_a only contains values measured or values assumed. z_a does not contain already calculated values. This is to minimize the effect of correlation in the error propagation. σ_{z_a} are the correspoinding errors of the starting or assumed values. Then $f_a(z_a) = \overline{a} = a$. Is the function that returns a when plugged in with z_a. We introduced this function to take its jacobian for the final formula of σ_a. When we write $J_{f_a}(z_a)^2$, we mean component wise squars. so $J_{f_a}(z_a)^2 = \left(\left(\partial_{z_{a,1}} f_a\right)^2, ..., ..., \left(\partial_{z_{a,n}} f_a\right)^2\right)$. The same goes for $\sigma^2_{z_a} = \left(\sigma^2_{z_{a,1}}, ..., ..., \sigma^2_{z_{a,n}}\right)$.

A.2.i Error of C

$$
\begin{aligned}
z_C &= (p_l, p_2, U_l, U_2) \\
\sigma_{z_C} &= \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}\right) \\
f_C(z_C) &= \frac{p_l - p_2}{U_l - U_2} \\
\sigma_C &= \sqrt{\left(J_{f_C}(z_C)^2\right)_{1\times 4} \left(\sigma^2_{z_C}\right)^T_{4\times 1}} = 0.02\frac{\text{bar}}{V}
\end{aligned}
\tag{1.1}
$$

A.2.ii Error of p_0

$$
\begin{aligned}
z_{p_0} &= (p_l, p_2, U_l, U_2) \\
\sigma_{z_{p_0}} &= \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}\right) \\
f_{p_0}\left(z_{p_0}\right) &= p_l - \frac{p_l - p_2}{1 - \frac{U_2}{U_l}} \\
\sigma_{p_0} &= \sqrt{\left(J_{f_{p_0}}\left(z_{p_0}\right)^2\right)_{1\times 4} \left(\sigma^2_{z_{p_0}}\right)^T_{4\times 1}}
\end{aligned}
\tag{1.2}
$$

A.2.iii Error of p_k

$$
\begin{aligned}
z_{p_k} &= (p_l, p_2, U_l, U_2, U_k) \\
\sigma_{z_{p_k}} &= \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}, \sigma_{U_k}\right) \\
f_{p_k}\left(z_{p_k}\right) &= p_l - \frac{p_l - p_2}{1 - \frac{U_2}{U_l}} + U_k \frac{p_l - p_2}{U_l - U_2} = \overline{p_0} + U_k \overline{C} \\
\sigma_{p_k} &= \sqrt{\left(J_{f_{p_k}}\left(z_{p_k}\right)^2\right)_{1\times 5} \left(\sigma^2_{z_{p_k}}\right)^T_{5\times 1}}
\end{aligned}
\tag{1.3}
$$

A.2.iv Error of p_e

$$
\begin{aligned}
z_{p_e} &= (p_l, p_2, U_l, U_2, U_e) \\
\sigma_{z_{p_e}} &= \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}, \sigma_{U_e}\right) \\
f_{p_e}\left(z_{p_e}\right) &= p_l - \frac{p_l - p_2}{1 - \frac{U_2}{U_l}} + U_e \frac{p_l - p_2}{U_l - U_2} = \overline{p_0} + U_e \overline{C} \\
\sigma_{p_e} &= \sqrt{\left(J_{f_{p_e}}\left(z_{p_e}\right)^2\right)_{1\times 5} \left(\sigma^2_{z_{p_e}}\right)^T_{5\times 1}}
\end{aligned}
\tag{1.4}
$$

A.2.v Error of t_0'

$$z_{t_0'} = (p_l, p_2, U_l, U_2, U_k, U_e, t_k)$$
$$\sigma_{z_{t_0'}} = \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}, \sigma_{U_k}, \sigma_{U_e}, \sigma_{t_k}\right)$$
$$f_{t_0'}\left(z_{t_0'}\right) = -t_k \frac{f_{p_e}\left(z_{p_e}\right)}{f_{p_k}\left(z_{p_k}\right) - f_{p_e}\left(z_{p_e}\right)} = -t_k \frac{\overline{p_e}}{\overline{p_k} - \overline{p_e}} \qquad \left(\text{we get } z_{p_e} \text{ and } z_{p_k} \text{ from } z_{t_0'}\right) \tag{1.5}$$
$$\sigma_{t_0'} = \sqrt{\left(J_{f_{t_0'}}\left(z_{t_0'}\right)^2\right)_{1\times 7} \left(\sigma^2_{z_{t_0'}}\right)^T_{7\times 1}}$$

A.2.vi Error of t_0

$$z_{t_0} = (p_l, p_2, U_l, U_2, U_k, U_e, t_k, t_e, t_l)$$
$$\sigma_{z_{t_0}} = \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}, \sigma_{U_k}, \sigma_{U_e}, \sigma_{t_k}, \sigma_{t_e}, \sigma_{t_l}\right)$$
$$f_{t_0}\left(z_{t_0}\right) = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

$$\varepsilon = 10^{-3}$$
$$\gamma = 10^{-5} \ /^{\circ}\mathrm{C}$$
$$a = (1+\varepsilon)\overline{p_e} - (1+\varepsilon+\gamma t_k)\overline{p_k} \tag{1.6}$$
$$b = \varepsilon(\overline{p_k} - \overline{p_e})t_k + (1+\gamma t_k)\overline{p_k} t_l - \overline{p_e}(t_l + t_k)$$
$$c = \overline{p_e} t_l t_k$$

$\left(\text{where } \overline{p_e} \text{ means } f_{p_e}\left(z_{p_e}\right) \text{ and } z_{p_e} \text{ is obtained from } z_{t_0} \quad \text{(the same goes for all overlines)}\right)$

$$\sigma_{t_0} = \sqrt{\left(J_{f_{t_0}}\left(z_{t_0}\right)^2\right)_{1\times 9} \left(\sigma^2_{z_{t_0}}\right)^T_{9\times 1}}$$

A.2.vii Error of $p_{\ln 2}$

$$z_{p_{\ln 2}} = (p_l, p_2, U_l, U_2, U_{\ln 2})$$
$$\sigma_{z_{p_{\ln 2}}} = \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}, \sigma_{U_{\ln 2}}\right)$$
$$f_{p_{\ln 2}}\left(z_{p_{\ln 2}}\right) = p_l - \frac{p_l - p_2}{1 - \frac{U_2}{U_l}} + U_{\ln 2}\frac{p_l - p_2}{U_l - U_2} = \overline{p_0} + U_{\ln 2}\overline{C} \tag{1.7}$$
$$\sigma_{p_{\ln 2}} = \sqrt{\left(J_{f_{p_{\ln 2}}}\left(z_{p_{\ln 2}}\right)^2\right)_{1\times 5} \left(\sigma^2_{z_{p_{\ln 2}}}\right)^T_{5\times 1}}$$

A.2.viii Error of $t_{\ln 2}'$

$$z_{t_{\ln 2}'} = (p_l, p_2, U_l, U_2, U_k, U_e, t_k, t_e, t_l, U_{\ln 2})$$
$$\sigma_{z_{t_{\ln 2}'}} = \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}, \sigma_{U_k}, \sigma_{U_e}, \sigma_{t_k}, \sigma_{t_e}, \sigma_{t_l}, \sigma_{U_{\ln 2}}\right)$$
$$f_{t_{\ln 2}'}\left(z_{t_{\ln 2}'}\right) = \left(1 - \frac{\overline{p_{\ln 2}}}{\overline{p_e}}\right)\overline{t_0} \tag{1.8}$$

$\left(\text{where } \overline{t_0} \text{ means } f_{t_0}\left(z_{t_0}\right) \text{ and } z_{t_0} \text{ is obtained from } z_{t_{\ln 2}'} \quad \text{(the same goes for all overlines)}\right)$

$$\sigma_{t_{\ln 2}'} = \sqrt{\left(J_{f_{t_{\ln 2}'}}\left(z_{t_{\ln 2}'}\right)^2\right)_{1\times 10} \left(\sigma^2_{z_{t_{\ln 2}'}}\right)^T_{10\times 1}}$$

A.2.ix Error of t_{ln2}

$$
\begin{gathered}
z_{t_{\mathrm{ln2}}} = (p_l, p_2, U_l, U_2, U_k, U_e, t_k, t_e, t_l, U_{\mathrm{ln2}}) \\
\sigma_{z_{t_{\mathrm{ln2}}}} = \left(\sigma_{p_l}, \sigma_{p_2}, \sigma_{U_l}, \sigma_{U_2}, \sigma_{U_k}, \sigma_{U_e}, \sigma_{t_k}, \sigma_{t_e}, \sigma_{t_l}, \sigma_{U_{\mathrm{ln2}}}\right) \\
f_{t_{\mathrm{ln2}}}\left(z_{t_{\mathrm{ln2}}}\right) = \frac{A\overline{t_0} + \overline{p_{\mathrm{ln2}}}}{A - \gamma\overline{p_{\mathrm{ln2}}}} \\
\\
\varepsilon = 10^{-3} \\
\gamma = 10^{-5}\ /^{\circ}\mathrm{C} \\
A = \frac{\overline{p_e}}{t_e - \overline{t_0}} + \frac{\varepsilon(\overline{p_e} - \overline{p_{\mathrm{ln2}}})}{t_l - \overline{t_0}}
\end{gathered}
\tag{1.9}
$$

$\left(\text{where } \overline{t_0} \text{ means } f_{t_0}\left(z_{t_0}\right) \text{ and } z_{t_0} \text{ is obtained from } z_{t_{\mathrm{ln2}}} \quad \text{(the same goes for all overlines)}\right)$

$$
\sigma_{t_{\mathrm{ln2}}} = \sqrt{\left(J_{f_{t_{\mathrm{ln2}}}}\left(z_{t_{\mathrm{ln2}}}\right)^2\right)_{1\times 10}\left(\sigma^2_{z_{t_{\mathrm{ln2}}}}\right)^T_{10\times 1}}
$$

Bibliography

[1] physikpraktikum 1, “manual absolute zero.” Accessed: Nov. 06, 2026. [Online]. Available: https://moodle-app2.let.ethz.ch/pluginfile.php/2575436/mod_folder/content/0/09_Manual.pdf?forcedownload=1

[2] SciPy, “jacobian.” Accessed: Nov. 06, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.differentiate.jacobian.html